

Entity Framework Core A Fondo

Dominando el Acceso a Datos en .NET 10 y ASP.NET Core

Hoja de Ruta

1. Introducción

- > Qué es un ORM
- > Ventajas vs SQL directo

2. Modelado

- > Entidades (C# 14) y DbContext
- > Data Annotations & Fluent API

3. Relaciones

- > 1 a Muchos
- > Muchos a Muchos

4. Consultas (LINQ)

- > Filtrado y DTOs
- > Carga Ansiosa y AsNoTracking

5. Mantenimiento

- > Operaciones CRUD
- > Bulk Ops (ExecuteUpdate)

6. Razor Pages

- > Constructores Primarios
- > Patrón PRG y Tag Helpers

Sección 1:

Introducción a los ORM

¿Por qué dejamos de escribir SQL a mano?

¿Qué es un **ORM**?

Object-Relational Mapping

Un ORM es una técnica de programación para convertir datos entre sistemas de tipos incompatibles utilizando lenguajes de programación orientados a objetos.

 **Base de Datos:** Tablas, Filas, Columnas.

 **C#:** Clases, Objetos, Propiedades.

 **El ORM:** Es el puente traductor entre ambos mundos.

El Paradigma



Tabla



Clase C#

SQL Directo vs EF Core

El Pasado (ADO.NET)

```
var lista = new List<Usuario>();
using var cmd = conn.CreateCommand();
cmd.CommandText = "SELECT Id, Nombre FROM Usuarios";

using var reader = cmd.ExecuteReader();
while (reader.Read())
{
    lista.Add(new Usuario {
        Id = reader.GetInt32(0),
        Nombre = reader.GetString(1)
    });
}
```

El Presente (.NET 10)

```
// EF Core se encarga de la conexión,
// el SQL asíncrono y el mapeo.

var lista = await db.Usuarios
    .AsNoTracking()
    .ToListAsync(ct);
```

Menos código = Menos errores. Además, EF Core nos protege de inyecciones SQL por defecto.

Ventajas de usar un **ORM**



Productividad

Reduce drásticamente el código repetitivo de acceso a datos (boilerplate). Te enfocas en la lógica de negocio real de la aplicación.



Tipado Fuerte

Los errores se detectan al compilar C#, no al ejecutar. Si cambias el nombre de una propiedad, el compilador te avisa de inmediato.



Agnosticismo

Puedes migrar de SQLite a SQL Server o PostgreSQL cambiando únicamente una línea de configuración; el código de consultas C# se mantiene intacto.

Sección 2:

Entidades y DbContext

Los cimientos de tu base de datos en C#

La Entidad POCO (C# 14)

Clases de Dominio

Las entidades son clases simples que representan una tabla. Con C# 14 utilizamos constructores limpios, `required`, la palabra clave `field` y expresiones de colección.

- ✓ `Id` es Primary Key por convención.
- ✓ `required` obliga a la inicialización.
- ✓ `[]` inicializa colecciones vacías (evita `NullRef`).

```
public class Autor
{
    public Guid Id { get; init; } = Guid.NewGuid();

    // Evitamos advertencias de nulos
    public required string Nombre { get; set; }

    // C# 14: Acceso directo al campo de respaldo (field)
    public string Email
    {
        get;
        set => field = value?.Trim().ToLowerInvariant()
            ?? throw new ArgumentNullException(nameof(value));
    }

    // Expresiones de colección modernas
    public List<Libro> Libros { get; set; } = [];
}
```


El Gestor Central: DbContext

```
using Microsoft.EntityFrameworkCore;

// C# 14: Constructor Primario inyectando opciones
public class AppDbContext(DbContextOptions<AppDbContext> options)
    : DbContext(options)
{
    // Cada DbSet representa una tabla consultable
    public DbSet<Autor> Autores => Set<Autor>();
    public DbSet<Libro> Libros => Set<Libro>();
}
```

Orquestando el ORM

El DbContext es la clase más importante. Responsabilidades:

-  Gestiona la conexión a la base de datos.
-  Rastrea los cambios en los objetos (Change Tracking).
-  Traduce las consultas LINQ a sentencias SQL nativas.

Sección 3:

Configuración del Modelo

Data Annotations, Fluent API y Complex Types

Limitaciones de las Convenciones

Por qué configurar

Por defecto, EF Core crea columnas NVARCHAR(MAX) para los strings y usa nombres de tablas pluralizados. En el mundo real, necesitamos imponer reglas (ej: Nombre no puede superar 100 caracteres, relaciones en cascada personalizadas).

Disponemos de dos enfoques principales y un nuevo tipo para estructuras (Complex Types).



Definiendo Reglas

Longitudes máximas, precisión de decimales, nulabilidad, nombres de tablas físicos y mapeo de relaciones complejas.

Enfoque 1: Data Annotations

Atributos Decorativos

Se importan de `System.ComponentModel.DataAnnotations`.
Útiles para validaciones automáticas en APIs o Razor Pages.

- + Rápido y visible al instante.
- Ensucia las clases de dominio puro.

```
[Table("Cat_Libros")] // Renombra la tabla
public class Libro
{
    [Key] // Fuerza a ser Primary Key
    public Guid Codigo { get; init; }

    [Required]
    [MaxLength(150)]
    public string Titulo { get; set; } = "";

    [Column(TypeName = "decimal(18,2)")]
    public decimal Precio { get; set; }
}
```


Enfoque 2: Fluent API

```
protected override void OnModelCreating(ModelBuilder mb)
{
    mb.Entity<Libro>(e =>
    {
        e.ToTable("Cat_Libros");
        e.HasKey(l => l.Codigo);

        e.Property(l => l.Titulo)
            .IsRequired()
            .HasMaxLength(150);

        e.Property(l => l.Precio)
            .HasPrecision(18, 2);
    });
}
```

El Método OnModelCreating

Ofrece mucha más potencia que los Data Annotations, manteniéndose en el DbContext.

- + Mantiene los POCOs totalmente limpios (Arquitectura Limpia).
- + Permite configuraciones imposibles con atributos (ej: claves compuestas).

Value Objects: Complex Types & JSON

Tipos Complejos (EF Core 9+)

Para objetos que no tienen identidad propia (sin Id), como una Dirección. Evita crear tablas innecesarias, mapeándolos en la misma tabla de forma plana o como JSON nativo.

```
[ComplexType]
public record Direccion(string Calle, string CP);

public class Usuario
{
    public Guid Id { get; init; }
    public required string Nombre { get; set; }

    // Se guardará en la tabla Usuario
    public required Direccion Domicilio { get; set; }
}

// Fluent API para forzar guardado en columna JSON:
// mb.Entity().OwnsOne(u => u.Domicilio,
//     b => b.ToJson());
```


Comparativa de Enfoques

Característica	Data Annotations	Fluent API
Ubicación	En la clase de la Entidad.	En el DbContext (o configuraciones separadas).
Legibilidad	Rápida, visible directamente al leer la propiedad.	Separada, requiere buscar en la infraestructura.
Potencia	Limitada (Reglas y validaciones básicas).	Total (Claves compuestas, índices, JSON, etc.).
Recomendación	Prototipos rápidos, validación de DTOs/Input en UI.	Proyectos grandes y arquitecturas limpias y desacopladas.

Sección 4:

Relaciones entre Entidades

Foreign Keys y Navigation Properties

Conceptos de Navegación



Clave Foránea (FK)

Es una propiedad de tipo primitivo (ej: Guid AutorId) que guarda el ID de la entidad relacionada en la tabla física SQL.



Prop. de Navegación

Es una referencia a la clase relacionada en C# (ej: Autor? Autor). Permite acceder a los datos sin tener que escribir JOINS manualmente.

Relación 1 a Muchos (1:N)

El Lado del "UNO"

```
public class Autor
{
    public Guid Id { get; init; }
    public required string Nombre { get; set; }

    // Colección para los MUCHOS
    public List<Libro> Libros { get; set; } = [];
}
```

El Lado Dependiente ("MUCHOS")

```
public class Libro
{
    public Guid Id { get; init; }
    public required string Titulo { get; set; }

    // Clave Foránea (Convención: ClaseId)
    public Guid AutorId { get; set; }

    // Propiedad de navegación (El UNO)
    public Autor? Autor { get; set; }
}
```


Relación Muchos a Muchos (N:M)

Mapeo Automático

Un Libro tiene muchas Categorías. Una Categoría tiene muchos Libros. EF Core genera la tabla intermedia SQL automáticamente si expones colecciones puras.

```
public class Categoria
{
    public Guid Id { get; init; }
    public required string Nombre { get; set; }

    public List<Libro> Libros { get; set; } = [];
}
```

En la clase Libro

```
public class Libro
{
    public Guid Id { get; init; }
    public required string Titulo { get; set; }

    // Colección inversa: EF hace la magia
    public List<Categoria> Categorías { get; set; } = [];
}
```


Relación 1 a 1

El Escenario

Un Usuario tiene un solo Perfil (datos privados/pesados). La clave foránea solo se coloca en la tabla dependiente para asegurar unicidad.

```
public class Usuario
{
    public Guid Id { get; init; }
    public required string Username { get; set; }

    public Perfil? Perfil { get; set; }
}
```

El Dependiente (FK única)

```
public class Perfil
{
    public Guid Id { get; init; }
    public required string Bio { get; set; }

    // La FK apunta al padre
    public Guid UsuarioId { get; set; }
    public Usuario? Usuario { get; set; }
}
```


Sección 5:

Tipos de Consultas (LINQ)

Leyendo datos de forma segura, asíncrona y eficiente

Consultas Básicas & AsNoTracking

Ejecución Diferida & Seguridad

EF Core traduce el código a SQL. La consulta no viaja a la BD hasta llamar a `.ToListAsync(ct)` o similar.

¡Regla de Arquitectura!

Toda consulta de **solo lectura** debe usar `AsNoTracking()`. Evita almacenar copias en memoria innecesariamente.

```
// Propagando el CancellationToken (ct)
public async Task<List<Libro>> GetCarosAsync(CancellationToken
{
    return await db.Libros
        .AsNoTracking() // ← Clave para rendimiento
        .Where(l => l.Precio > 50)
        .OrderByDescending(l => l.Precio)
        .ToListAsync(ct);
}
```


Proyecciones a DTOs (Records)

Evitando el Over-fetching

Nunca descargues una entidad pesada si solo quieres pintar su título. Proyecta usando un record immutable y `.Select()`.

```
// DTO Posicional immutable
public record LibroDto(Guid Id, string Titulo, decimal Precio);
```

La Consulta Optimizada

```
var catalogo = await db.Libros
    .AsNoTracking()
    .Select(l => new LibroDto(l.Id, l.Titulo, l.Precio))
    .ToListAsync(ct);

// SQL Generado (No trae todas las columnas):
// SELECT [l].[Id], [l].[Titulo], [l].[Precio] FROM [Libros]
```


Agregación & Paginación

Matemáticas en Base de Datos

```
// Cuenta registros en SQL, no en RAM  
int total = await db.Libros.CountAsync(ct);  
  
// Suma directa en SQL Server / SQLite  
decimal inventario = await db.Libros  
    .SumAsync(l => l.Precio, ct);
```

Paginación Servidor (.Skip .Take)

```
// Devuelve la página 2 con 20 elementos  
var paginaDos = await db.Libros  
    .AsNoTracking()  
    .OrderBy(l => l.Titulo)  
    .Skip(20) // Omite los primeros 20  
    .Take(20) // Toma los siguientes 20  
    .ToListAsync(ct);
```


Carga Ansiosa (.Include)

El Problema (Null Trap)

Si pides un Libro, EF Core no trae el Autor por defecto para ahorrar datos.

```
var libro = await db.Libros.FirstAsync(ct);  
  
// ❌ libro.Autor es NULL aquí (Falla)  
Console.WriteLine(libro.Autor.Nombre);
```

La Solución (Eager Loading)

Fuerza el JOIN usando .Include() explícitamente.

```
var libro = await db.Libros  
    .AsNoTracking()  
    .Include(l => l.Autor) // SQL LEFT JOIN  
    .FirstAsync(ct);  
  
// ✅ Funciona correctamente  
Console.WriteLine(libro.Autor!.Nombre);
```


El Temido Problema $N + 1$

El Error Común (Lento)

Hacer consultas dentro de bucles destroza la BD.

```
// 1 Consulta principal
var autores = await db.Autores.ToListAsync(ct);

foreach(var a in autores) {
    // N Consultas extra! (Lentísimo)
    var num = db.Libros.Count(l => l.AutorId == a.Id);
}
```

La Solución Proyectada

Resolver todo en un solo viaje usando proyección de navegación.

```
// Todo en 1 sola ida a la base de datos
var datos = await db.Autores
    .AsNoTracking()
    .Select(a => new {
        Nombre = a.Nombre,
        TotalLibros = a.Libros.Count()
    })
    .ToListAsync(ct);
```


Sección 6:

Operaciones de Escritura

Añadiendo, modificando y borrando datos

El Patrón Unidad de Trabajo



1. Preparar

Usas `.Add()`, `.Update()`, o `.Remove()` para decirle a EF Core lo que quieres hacer en la memoria.



2. Tracker

El `DbContext` "rastrea" todos estos cambios de estado internamente. Aún no ha ejecutado ninguna escritura en SQL.



3. SaveChanges

Al invocar `.SaveChangesAsync()`, EF genera todo el SQL y lo envía en una única y segura transacción atómica.

Insertar y Actualizar (Tracking)

Insertar (Create)

```
var autor = new Autor { Nombre = "Tolkien" };

// Estado = Added
db.Autores.Add(autor);

// Genera: INSERT INTO Autores ...
await db.SaveChangesAsync(ct);
```

Actualizar (Change Tracking)

```
// Al buscarlo SIN AsNoTracking, EF lo vigila
var autor = await db.Autores.FindAsync([1], ct);

if (autor != null) {
    // Cambiamos la propiedad. Estado = Modified
    autor.Nombre = "J.R.R. Tolkien";

    // Genera: UPDATE Autores SET Nombre ...
    await db.SaveChangesAsync(ct);
}
```


Rendimiento: Operaciones Masivas

Bulk Delete (EF Core 9+)

En lugar de traer registros a RAM para luego borrarlos, los borramos directamente en SQL con `ExecuteDeleteAsync()`.

```
// DELETE FROM Libros WHERE Precio < 10  
int borrados = await db.Libros  
    .Where(l => l.Precio < 10m)  
    .ExecuteDeleteAsync(ct);
```

Bulk Update

Sube el precio sin consultar entidades previas.

```
int actualizados = await db.Libros  
    .Where(l => l.AutorId == targetId)  
    .ExecuteUpdateAsync(setters => setters  
        .SetProperty(l => l.Precio, l => l.Precio * 1.10m),  
    ct);
```


Sección 7:

Integración Web

Inyección de dependencias y Razor Pages

Configuración en Program.cs

```
var builder = WebApplication.CreateBuilder(args);

// Pool: Reutiliza contextos para alto rendimiento
builder.Services.AddDbContextPool<AppDbContext>(opt =>
    opt.UseSqlite(
        builder.Configuration.GetConnectionString("Db")
    )
);

builder.Services.AddRazorPages();

var app = builder.Build();
app.MapRazorPages();
app.Run();
```

Inyección de Dependencias

En ASP.NET Core no instanciamos `new AppDbContext()`.
Registramos un "Pool" en el contenedor de Inversión de Control.

Esto permite a cualquier página o servicio solicitar el `DbContext` por constructor, manteniendo el ciclo de vida gestionado de forma automática y reciclable.

El PageModel Moderno (C# 14)

Controlando el Ciclo HTTP

Usamos Constructores Primarios para inyectar, y aplicamos el patrón PRG tras peticiones POST para evitar reenvíos de formulario.

</> C# 14 Primary Constructors

✓ ModelState.IsValid

↺ RedirectToPage()

```
// Inyección limpia mediante constructor primario
public class CreateModel(AppDbContext db) : PageModel
{
    [BindProperty]
    public required CreateLibroInput Input { get; set; }

    public async Task<IActionResult> OnPostAsync(CancellationToken)
    {
        if (!ModelState.IsValid) return Page();

        db.Libros.Add(new Libro { Titulo = Input.Titulo });
        await db.SaveChangesAsync(ct);

        // Patrón PRG (Post-Redirect-Get)
        return RedirectToPage("./Index");
    }
}
```


La Vista **Razor** (Tag Helpers)

```
@page
@model CreateModel

method="post">

    asp-for="Input.Titulo">

    asp-for="Input.Titulo" class="form-control" />

    asp-validation-for="Input.Titulo"
        class="text-danger">

    type="submit">Guardar
```

Cero Lógica C# Compleja

Mantenemos las vistas `.cshtml` limpias. Utilizamos los **Tag Helpers** de ASP.NET (`asp-for`, `asp-validation-for`) para la generación declarativa de HTML en lugar del estilo heredado de MVC.

Cualquier lista u objeto requerido por la vista debe ser inyectado/proyectado en el `PageModel` previamente.

¿Preguntas?

Hemos dominado las bases del ORM con código limpio y moderno.

`</>` Masterclass de Desarrollo .NET 10